Pruebas unitarias para la kata Tres en Raya

Suite del repositorio (TDD) frente a pruebas generadas por IA

Franco Parra

Octubre de 2026

Gestión de Calidad de Sistemas · Universidad Católica Boliviana «San Pablo»

Contenido

1. Contexto

2. Metodología

3. Resultados

4. Diferencias sustanciales

5. Conclusiones

Contexto

La kata del Tres en Raya

R1 · Tablero de 3x3

- Fuera del eje X ⇒ excepción
- Fuera del eje Y ⇒ excepción
- Casilla ocupada ⇒ excepción

R2 · Turnos

- El primer turno es de **X**
- Después de X juega **O**
- Después de O juega X

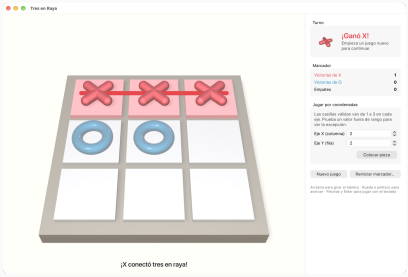
R3 · Ganador

- Sin línea completa no hay ganador
- Gana con una línea horizontal
- Gana con una línea vertical
- Gana con una diagonal

Diez pruebas definidas por la kata. El objetivo del trabajo es comparar cómo las cubren una suite escrita con TDD y dos suites generadas por una IA a partir de un prompt.

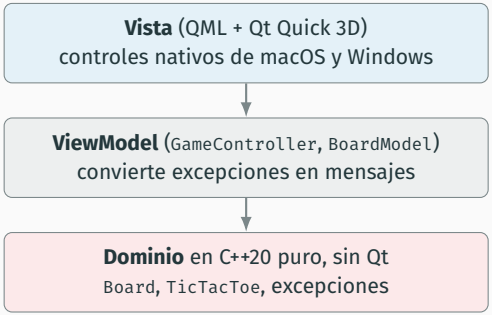
Entregables

- **Repositorio público:** `github.com/SkyWing3/tres-en-raya`
- **Prompts:** `ia/prompts.md`
- **Respuestas de la IA, sin cambios:** `ia/respuestas/`
- **Mediciones reproducibles:** `tools/compare_suites.py` → `docs/resultados/`
- **Esta presentación:** `docs/presentacion/`
- **Integración continua:** GitHub Actions en Linux, macOS y Windows



Aplicación Qt con tablero en 3D

La solución: dominio separado de la interfaz



- Las reglas de la kata viven en el dominio y se prueban sin interfaz.
- Excepciones con datos: `OutOfBoardException` informa el eje y el valor.
- Una jugada inválida no modifica la partida.
- Qt Test, CMake con presets y advertencias como errores (`-Werror`).

Metodología

Tres suites de pruebas

Suite	Cómo se obtuvo	Ubicación
Repositorio (TDD)	Escrita con TDD junto con el código, compilando y ejecutando en cada paso, con asistencia de Claude Code. Incluye el ViewModel.	`tests/tictactoe/` `tests/presentation/`
IA básica	Respuesta de la IA al prompt 1: sólo los requerimientos de la kata, sin el código.	`tests/ia/basico/`
IA estructurada	Respuesta de la IA al prompt 2: rol, contexto técnico, API real y reglas de calidad.	`tests/ia/estructurado/`

- Las suites de IA se generaron con **Claude** (Anthropic) en una sola respuesta y se compilaron sin modificarlas, con las mismas advertencias estrictas.
- La suite básica supuso su propia API: se ejecutó mediante un **adaptador** de unas 30 líneas escrito por el equipo.

Prompt 1: básico

```
Genera las pruebas unitarias en C++ usando Qt Test para la
siguiente kata de TDD de Tres en Raya:

Requerimiento 1: Una pieza puede estar colocada en un espacio
de un tablero 3x3
Pruebas:
- Cuando una pieza está fuera del eje X, entonces se debe
  lanzar una excepción
- ...
Requerimiento 2: Debe existir una forma de determinar qué
jugador juega después
- ...
Requerimiento 3: Un jugador gana si es el primero en conectar
una línea de un lado a otro, o de una esquina a otra
- ...
```

Estrategia: lo que escribiría un estudiante que copia el enunciado.

Resultado:

- 11 pruebas, una por cada prueba de la kata
- Nombres *given/when/then*
- Inventa la API: `play()` devuelve un texto

Prompt 2: estructurado

```
Actúa como un ingeniero de software especializado en pruebas
unitarias para C++.
## Contexto
- C++20, Qt 6.10+, Qt Test, -Werror ...
- El dominio YA está implementado. Esta es su API pública:
  class TicTacToe { GameStatus play(Position); ... };
  class OutOfBoardException { Axis axis(); int value(); };
## Requerimientos de la kata ...
## Reglas para las pruebas
3. Verifica el TIPO concreto de la excepción y sus datos.
4. Usa pruebas guiadas por datos para las 3 filas, las 3
   columnas y las 2 diagonales.
5. Incluye casos límite: 0 y 4, negativos, empate, victoria
   en la última jugada, jugar después de terminar...
```

Añade: rol, contexto técnico, la API real y reglas de calidad.

Resultado:

- 23 casos, compila a la primera
- Excepciones tipadas y pruebas guiadas por datos
- Una prueba defectuosa

Cómo se midió

Métricas

- Casos ejecutados, aserciones y líneas de prueba
- **Cobertura** de líneas y ramas del dominio con `llvm-cov`
- **Puntaje de mutación**: porcentaje de errores introducidos a propósito que la suite detecta

Todo se calcula con `tools/compare_suites.py` sobre una copia temporal del proyecto. Cada prueba se ejecuta en su propio proceso.

Ejemplo de mutante (M3)

```
// Original
return coordinate >= kMinCoordinate
    && coordinate <= kMaxCoordinate;
// Mutante: ahora acepta la coordenada 4
return coordinate >= kMinCoordinate
    && coordinate <= kMaxCoordinate + 1;
```

Si alguna prueba falla, el mutante **muere** (se detectó el error). Si todas pasan, **sobrevive**.

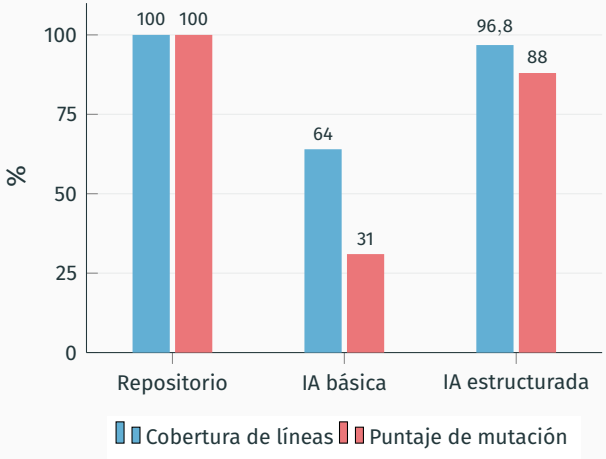
Resultados

Resumen de las tres suites

Métrica	**Repositorio (TDD)**	**IA: prompt básico**	**IA: prompt estructurado**
Casos ejecutados	85	11	23
Fallan con el código correcto	0	0	1
Aserciones	165	11	40
Líneas de código de prueba	699	103	194
Cobertura de líneas (dominio)	100,0 %	64,0 %	96,8 %
Cobertura de ramas (dominio)	100,0 %	83,3 %	100,0 %
Mutantes detectados	16 de 16	5 de 16	14 de 16
Puntaje de mutación	100 %	31 %	88 %

Cobertura medida sobre el dominio (`src/core`). La suite del repositorio además prueba el ViewModel, con 99,6 % de líneas cubiertas en todo el proyecto.

Cobertura no es lo mismo que calidad



- La IA estructurada cubre el 96,8 % de las líneas, pero deja vivos dos mutantes.
- Ejecutar una línea no es lo mismo que verificar su resultado.
- El contexto en el prompt eleva el puntaje de 31 % a 88 %.

Mutantes detectados por cada suite

	Req.	Error introducido	**Repo**	**IA básica**	**IA estruct.**
M1	R1	Rechaza la coordenada 3 (límite superior excluido)	✓	✓	✓
M2	R1	Acepta la coordenada 0	✓	✗	✓
M3	R1	Acepta la coordenada 4	✓	✗	✓
M4	R1	Informa el eje equivocado al salir del eje X	✓	✗	✓
M5	R1	Permite jugar sobre una casilla ocupada	✓	✓	✓
M6	R1	El tablero nunca se considera lleno	✓	✗	✓
M7	R2	El primer turno es de O	✓	✓	✓
M8	R2	Los turnos no alternan	✓	✓	✓
M9	R2	Una jugada inválida consume el turno	✓	✗	✗
M10	R3	No reconoce la diagonal secundaria	✓	✓	✓
M11	R3	No reconoce la tercera columna	✓	✗	✓
M12	R3	No reconoce la fila inferior	✓	✗	✓
M13	R3	Un tablero lleno es empate aunque haya línea	✓	✗	✗
M14	R3	Se puede seguir jugando después de ganar	✓	✗	✓
M15	R3	winner() informa un ganador cuando nadie ganó	✓	✗	✓
M16	R3	reset() no reinicia el estado de la partida	✓	✗	✓

✓ detectado ✗ sobrevive

Diferencias sustanciales

1. API inventada frente a API real

IA básica: supone una API propia

```
class TicTacToe {
public:
    // "No hay ganador" o "X es el ganador"
    QString play(int x, int y);
    char nextPlayer() const;
};
QCOMPARE(actual, QString("X es el ganador"));
```

Repositorio: tipos del dominio

```
GameStatus play(Position position);
Player nextPlayer() const noexcept;
std::optional<Player> winner() const;
std::optional<Line> winningLine() const;

QCOMPARE(status, GameStatus::Won);
QCOMPARE(m_game.winningLine(),
         (Line{{1, y}, {2, y}, {3, y}}));
```

- Sin conocer el código, la IA no puede compilar contra el proyecto: se necesitó un adaptador.
- Comparar textos es frágil: un cambio de redacción rompe las pruebas sin que cambie el comportamiento.
- El repositorio verifica también *qué* línea ganó, no sólo que alguien ganó.

2. Excepciones genéricas frente a excepciones tipadas

IA básica

```
QVERIFY_THROWS_EXCEPTION(std::runtime_error,
                         ticTacToe->play(5, 2));
```

- Cualquier error pasa la prueba: fuera del eje X, del eje Y u ocupada.
- Mutante M4 (informa el eje equivocado): ✗

Repositorio

```
try {
    m_game.play({x, 2});
    QFAIL("Se esperaba OutOfBoardException");
} catch (const OutOfBoardException &exception) {
    QCOMPARE(exception.axis(), Axis::X);
    QCOMPARE(exception.value(), x);
}
```

- Verifica el tipo exacto, el eje y el valor rechazado.
- Mutante M4: ✓

3. Valores límite y pruebas guiadas por datos

IA básica: un solo valor, lejos del límite

```
ticTacToe->play(5, 2);  // eje X
ticTacToe->play(2, 5);  // eje Y
```

Si el juego aceptara el 0 o el 4 (M2, M3), las pruebas seguirían pasando: ✗

Repositorio: `_data()` + `QFETCH`

```
QTest::newRow("x = 0") << 0;
QTest::newRow("x = 4") << 4;
QTest::newRow("x = -1") << -1;
QTest::newRow("x = INT_MIN")
    << std::numeric_limits<int>::min();
QTest::newRow("x = INT_MAX")
    << std::numeric_limits<int>::max();
```

Una sola función de prueba recorre cada valor límite y los extremos de `int`.

4. ¿Se prueban todas las formas de ganar?

Líneas ganadoras probadas	Repositorio	IA básica	IA estructurada
Filas	3 de 3	1 de 3	3 de 3
Columnas	3 de 3	1 de 3	3 de 3
Diagonales	2 de 2	2 de 2	2 de 2
Gana **O**	✓	✓	✗
Verifica *qué* línea ganó	✓	✗	✓

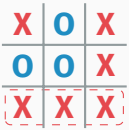
- La kata dice «*una* línea horizontal»; la IA básica la tomó literalmente y probó sólo la fila superior. Si el juego no reconociera la fila inferior o la tercera columna (M11, M12), no lo notaría.
- Las pruebas guiadas por datos cubren las 8 líneas sin duplicar código.

5. Una prueba defectuosa generada por la IA

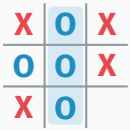
```
void TestTicTacToeIA::winOnLastMoveIsNotDraw()
{
    // X O X
    // O O X
    // X X X  <- X completa la fila 3 en la novena jugada
    playSequence({{1, 1}, {2, 1}, {3, 1}, {1, 2},
                  {3, 2}, {2, 2}, {1, 3}, {2, 3}});
    QCOMPARE(m_game.play({3, 3}), GameStatus::Won);
```

- El dibujo del comentario no corresponde a la secuencia de jugadas.
- La novena jugada lanza `GameOverException` y Qt Test aborta todo el ejecutable (código 134).
- Era la prueba que debía detectar M13, que ✗.

Lo que dice el comentario



Lo que se juega



En la octava jugada **O** ya completó la columna central.

6. Una prueba que no puede fallar

IA estructurada

```
m_game.play({1, 1}); // X
QVERIFY_THROWS_EXCEPTION(OccupiedCellException,
                         m_game.play({1, 1}));
QVERIFY_THROWS_EXCEPTION(OutOfBoardException,
                         m_game.play({4, 4}));
QCOMPARE(m_game.nextPlayer(), Player::O);
```

Con el error M9 (una jugada inválida consume el turno), cada jugada inválida cambia el turno: X $\rightarrow$ O $\rightarrow$ X… Dos cambios seguidos se anulan y la prueba **pasa**: ✗

Repositorio (corregido)

```
QVERIFY_THROWS_EXCEPTION(InvalidMoveException,
                         m_game.play({1, 1}));
QCOMPARE(m_game.nextPlayer(), Player::O);

QVERIFY_THROWS_EXCEPTION(InvalidMoveException,
                         m_game.play({0, 1}));
QCOMPARE(m_game.nextPlayer(), Player::O);
```

El repositorio tenía la misma debilidad: el análisis de mutantes la reveló y se corrigió verificando después de *cada* jugada.

7. Casos que la kata no pide, pero el juego necesita

Caso	Repositorio	IA básica	IA estructurada
Empate con el tablero lleno	✓	✗	✓
Victoria en la última jugada (no es empate)	✓	✗	~ defectuosa
No se puede jugar después de terminar	✓	✗	✓
Una jugada inválida no cambia el turno	✓	✗	~ enmascarada
Reiniciar después de ganar	✓	✗	✗
Extremos de `int` (`INT_MIN`, `INT_MAX`)	✓	✗	✗
Pruebas del tablero (`Board`) de forma aislada	✓	✗	✗
ViewModel: señales, marcador y mensajes para la vista	✓	✗	✗

La IA sólo agrega casos límite cuando el prompt los nombra uno por uno. TDD los hace aparecer al diseñar cada comportamiento.

8. Diseño y mantenibilidad de las pruebas

	Repositorio	**IA básica**	**IA estructurada**
Organización	Un archivo por requerimiento, más `Board` y ViewModel	Un archivo	Un archivo
Preparación (*fixture*)	Objeto por valor y `init()`	`new`/`delete` manuales	Objeto por valor y `init()`
Trazabilidad	Comentario que cita cada prueba de la kata	Agrupadas por requerimiento	Comentario que cita cada prueba
Diagnóstico al fallar	`QFAIL`/`QVERIFY2` con mensaje	Sin mensajes	`QFAIL` con mensaje
Verificación en compilación	`static_assert` de jugadores y primer turno	No	No
Aserciones por caso	1,9	1,0	1,7

Lo que la IA hizo bien

- **Rapidez**: una suite completa y legible en segundos.
- **Estructura correcta**: una prueba por caso de la kata, nombres descriptivos y preparación independiente de cada prueba.
- Con el prompt estructurado:
  - compiló a la primera con `-Werror` y contra la API real;
  - usó excepciones tipadas y pruebas guiadas por datos;
  - alcanzó 96,8 % de cobertura y 88 % de puntaje de mutación.
- El prompt cambia el resultado más que la herramienta: la calidad depende de lo que se le pide.

Conclusiones

Integración continua



- GitHub Actions, como el flujo con Maven del repositorio de referencia, pero en **Linux, macOS y Windows**.
- Cada `push` y *pull request* ejecuta las pruebas del repositorio; si alguna falla, el cambio queda marcado en rojo.
- Las suites de IA se ejecutan aparte: se conservan tal como se entregaron, con su prueba defectuosa.

Conclusiones

1. Con sólo el enunciado, la IA produce pruebas que parecen completas pero detectan 5 de 16 errores: cubren la letra de la kata, no el comportamiento.
2. Con contexto y reglas, la IA se acerca mucho (14/16), pero entregó una prueba incorrecta y otra que no podía fallar.
3. La cobertura no basta: 96,8 % de líneas cubiertas con dos errores sin detectar. Las **pruebas de mutación** miden mejor la calidad.
4. La diferencia de fondo es el **proceso**: escribir, ejecutar, medir y revisar cada prueba. Incluso así, el análisis encontró una debilidad en la suite del repositorio.

Recomendaciones para usar IA al escribir pruebas

En el prompt

- Incluir la API real (encabezados) y el marco de pruebas.
- Pedir excepciones tipadas y sus datos.
- Nombrar los casos límite que importan.
- Pedir pruebas guiadas por datos.

Después de generar

- Compilar y ejecutar *antes* de aceptar.
- Revisar que cada prueba verifique lo que dice su nombre.
- Medir cobertura **y** puntaje de mutación.
- Integrarlas a CI como cualquier otro código.

Declaración de uso de IA

- El código del repositorio y su suite de pruebas se desarrollaron con TDD con la asistencia de **Claude Code** (Anthropic), compilando, ejecutando y revisando cada cambio.
- Las suites «IA básica» e «IA estructurada» se generaron con **Claude** a partir de los prompts de `ia/prompts.md`, en una sola respuesta y sin correcciones.
- Al venir del mismo modelo, la comparación aísla el efecto del *proceso* (iterar y medir frente a una sola respuesta) más que el de la herramienta. Los prompts pueden repetirse en ChatGPT u otra IA.
- Todas las cifras se obtienen con herramientas reproducibles: `tools/compare_suites.py` y `tools/coverage.sh`.

Referencias

- K. Beck. *Test-Driven Development: By Example*. Addison-Wesley, 2002.
- V. Farcic y A. Garcia. *Test-Driven Java Development*. Packt, 2015 (kata del Tres en Raya).
- Y. Jia y M. Harman. «An Analysis and Survey of the Development of Mutation Testing». *IEEE Transactions on Software Engineering*, 37(5), 2011.
- Repositorio de referencia del curso: `https://github.com/gpalabral/TDD`
- Qt Test: `https://doc.qt.io/qt-6/qttest-index.html`
- Clang Source-based Code Coverage: `https://clang.llvm.org/docs/SourceBasedCodeCoverage.html`
- Repositorio de este trabajo: `https://github.com/SkyWing3/tres-en-raya`

¡Gracias!

¿Preguntas?